

CSE211 Web Programming

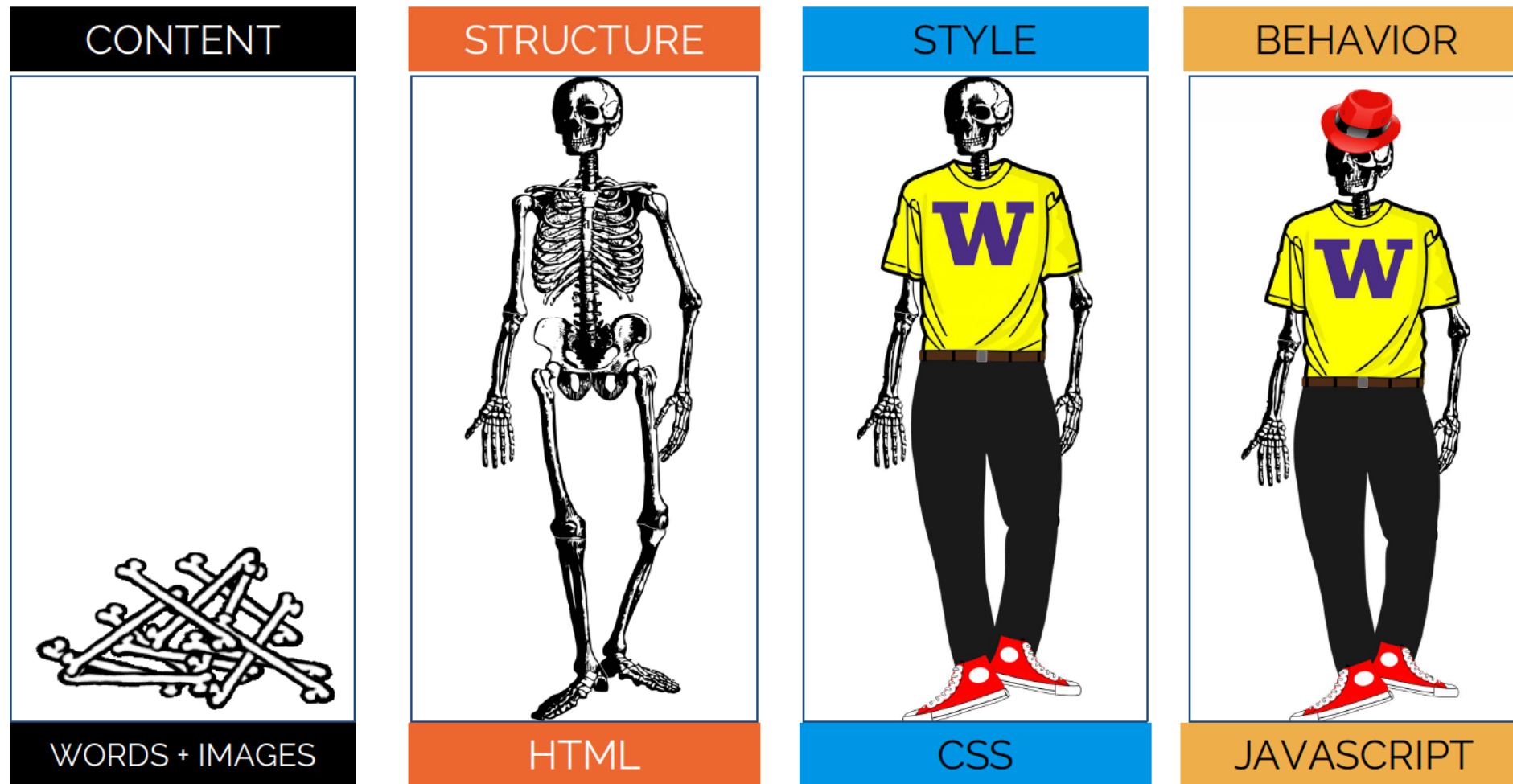
Lecture 05: HTML Basics

Topics:

- Front-End Foundations:
 - HTML5 Basic Page Structure
 - HTML5 elements
 - HTML Document head <head>
 - HTML Document body
 - Block-level elements
 - Text-level elements
 - Logical Text Formatting Elements
 - Attributes & Attribute Rules
 - HTML Basics.

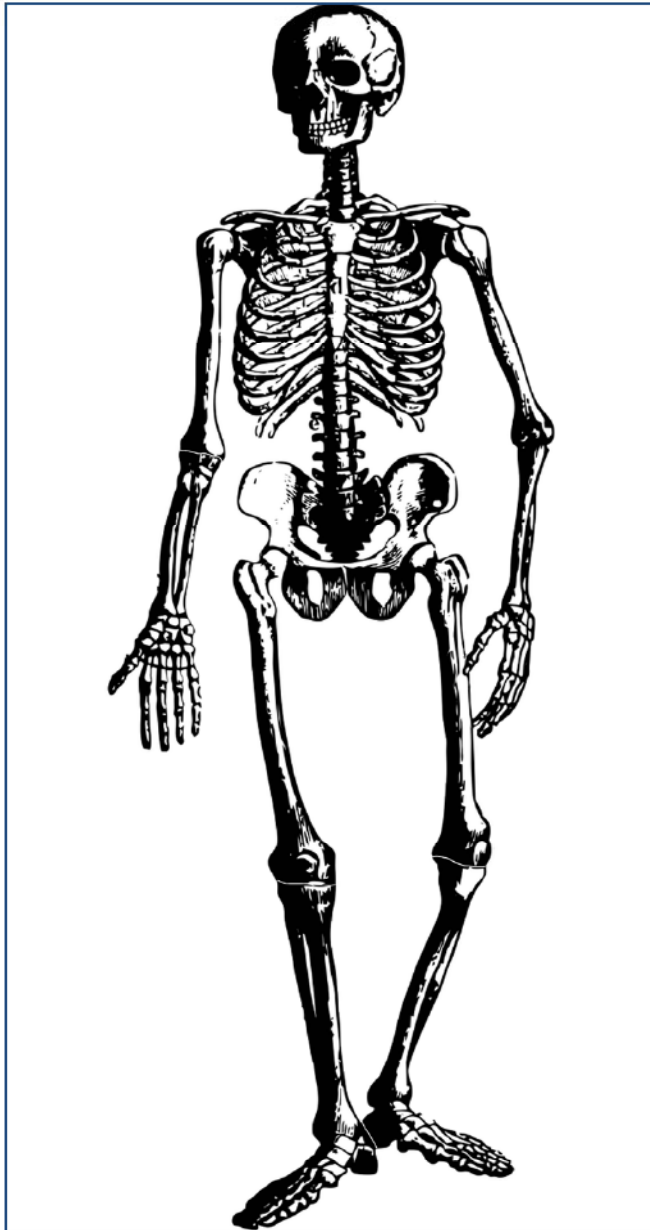


What is a Web Page?



- **Hypertext Markup Language ([HTML](#))**: semantic markup for web page content
 - **Cascading Style Sheets ([CSS](#))**: styling web pages
 - **Client-side [Javascript](#)**: adding programmable interactivity to web pages
 - **Asynchronous JavaScript and JSON (AJAX / Fetch API)**: fetching data from web services using [JavaScript fetch API](#)
- Modern Trend: Component-based frameworks (React, Vue), combine HTML, CSS, and JS in single components**

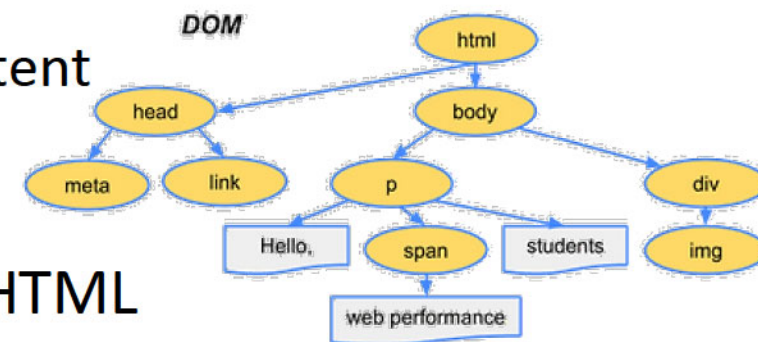
HTML



- ❑ **Hyper:** over/above/beyond
- ❑ **Text:** words and/or alphanumeric characters
- ❑ **Markup:** the result of preparing text or indicating the relationship between parts of text before displaying
- ❑ **Language:** a system of symbols used to communicate ideas
HTML is the set of symbols used to give text, images and other content additional meaning.



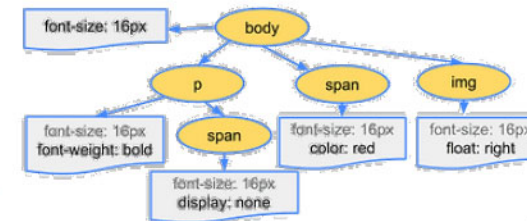
HTML5: Newest version of HTML/HTML



Cascaded Style Sheet (CSS)



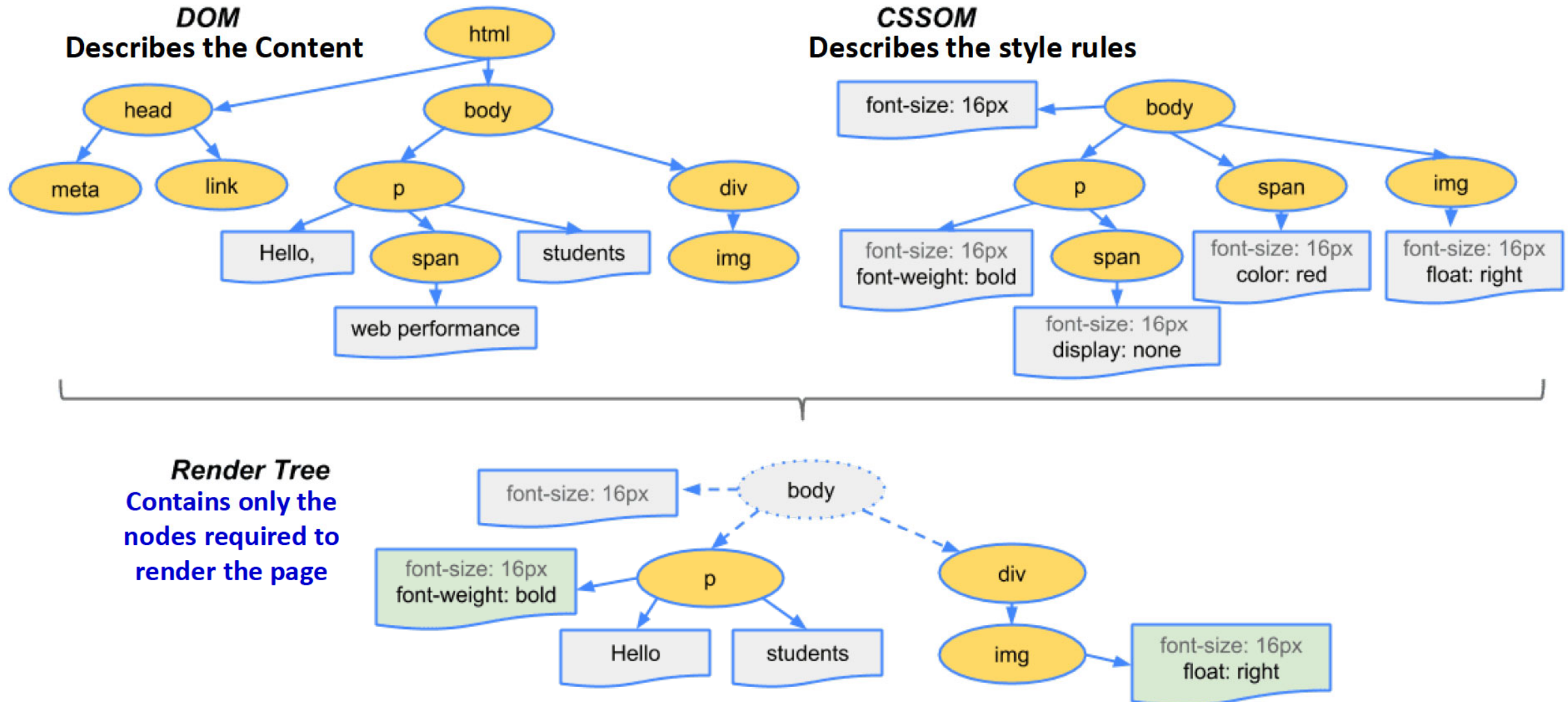
- ❑ **CSS**: is a simple mechanism/**stylesheet language** for **adding style** (e.g., fonts, colors, spacing, ... etc.) to Web documents.
 - **Cascading** refers to the order in which styles are applied
- ❑ **CSS**: describes how HTML elements should be displayed.
- ❑ **CSS**: generally used to describe a web page's **appearance/presentation**
- ❑ **CSS**: describes how elements should be **rendered** across devices, supporting responsive design through media queries.



DOM → CSSOM → Render → Paint pipeline



Render Tree



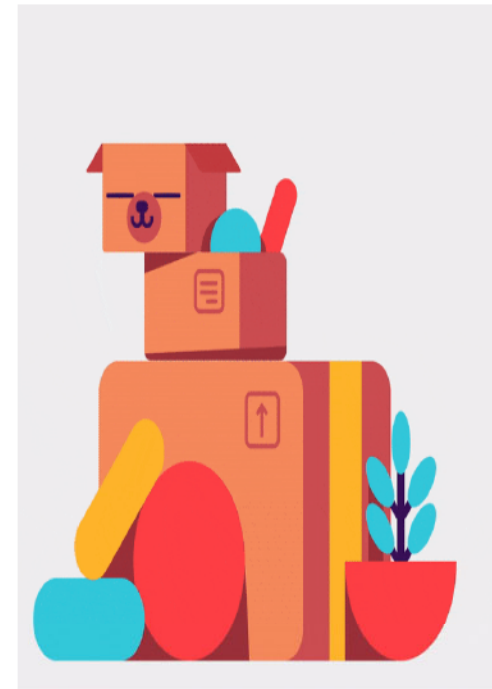
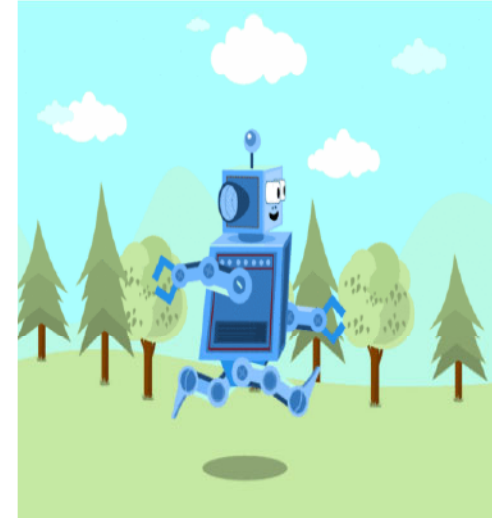
DOM → CSSOM → Render → Paint pipeline

JavaScript (JS)



- ❑ **JS:** is a high-level, event-driven programming language executed by the browser's JavaScript engine (like V8 or SpiderMonkey).
 - Runs in the browser (client-side)
 - Can also run on servers (Node.js)
 - Just-in-Time (JIT) compiled for performance
- ❑ **JS:** provide **dynamic interactions** and **asynchronous updates** that enhance user experience:
 - updating content without page reload,
 - controlling multimedia, animating images,
 - Form validation, Adding special effects and interactions.
- ❑ **Modern JS:** European Computer Manufacturers Association (ECMA)
 - ES6+ features (2015-present) are now standard
 - ❑ Arrow functions, async/await, modules, etc.

HTML → DOM → **JS updates nodes in real time**



Modern Web Development Stack (2024-2025)

□ Fundamentals (What we'll learn first):

- HTML → Structure
- CSS → Style
- JS → Behavior

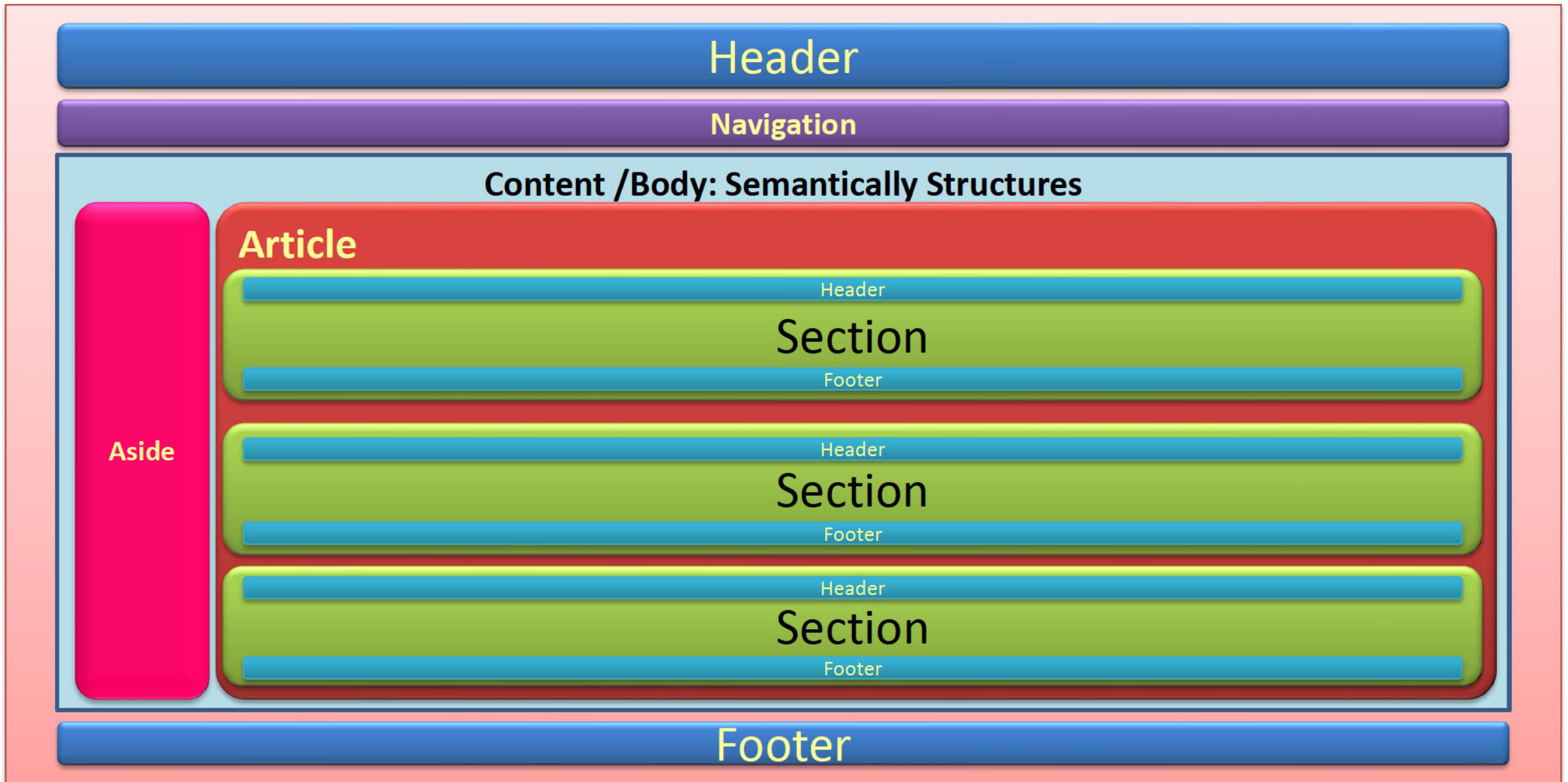
□ Modern Enhancements:

- **Build Tools:** Vite, Webpack
- **CSS Frameworks:** Tailwind CSS, Bootstrap 5
- **JS Frameworks:** React, Vue, Svelte
- **Package Managers:** npm, yarn
- **Backend:** PHP, Node.js, Express

□ **This Course Focus:** Master the fundamentals first (HTML/CSS/JS) as foundation for everything.

□ **Career Path:** After mastering basics, learn frameworks (we will briefly go through them)

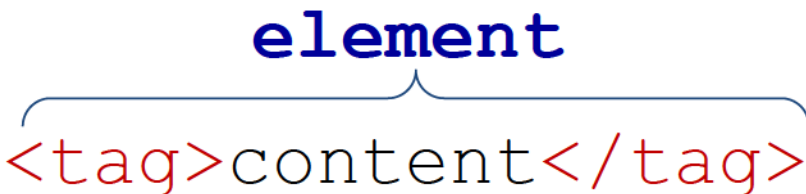
HTML5 Basic Page Structure



HTML5 improves accessibility, SEO, and structure clarity through semantic elements.

Hypertext Markup Language (HTML)

- Describes the content and structure of information on a web page
- Consists of **elements** each is defined with **start** and **end TAGs**.
- HTML is **case-insensitive**, but lowercase is standard.
- Most *whitespace* is insignificant in HTML (ignored or collapsed to a single space).

SYNTAX:  `<tag>content</tag>`

EXAMPLE: `<p>This is a paragraph</p>`

There are many different types of HTML tags used to structure web pages (we can't possibly cover all of them within class).

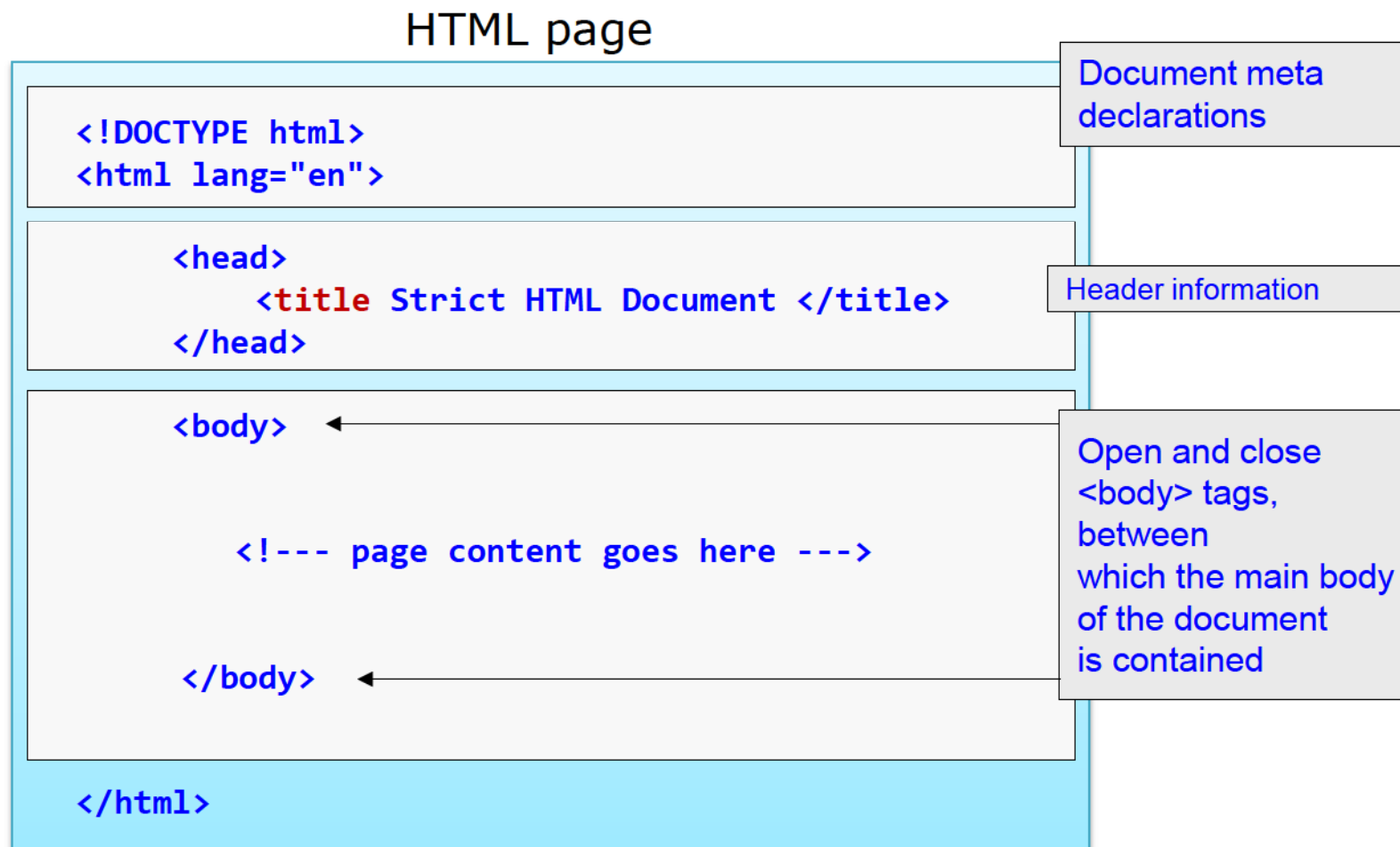
You can explore a complete tag reference on MDN Web Docs.

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element>

How HTML5 Documents are Structured

□ HTML documents are comprised of a simple three-part framework:

1. Document declaration
2. Header section
3. Body of document.



An HTML page is saved into a file ending with extension **.html**

The `<!DOCTYPE html>` tells the browser to interpret the page's code as **HTML5**, the latest version of the lang.

Earlier HTML versions used complex DTDs;

HTML5 replaced them with a single, simple declaration:

`<!DOCTYPE html>`

The `<head>` section contains metadata, links to stylesheets/scripts, and the document title, it doesn't render visually.

The `<body>` contains the page's content

Basic HTML page structure



Every HTML5 document must start with **<!DOCTYPE html>**
and should specify a **character set** and **viewport** for accessibility and responsive design.

HTML Document head <head>

<title> optional

- Names a web page, usually appears in the colored bar (called the title bar) at the top of the browser window
- Is the text identifying a page when users add your page to their list of Favorites or **Bookmarks**

<link> optional

Defines the relationship(s) between this document and an external resource. A document can have several <link> elements.

<meta> MUST

A container for document *meta-information*.

The **META** element can be used to identify properties of a document (e.g., author, expiration date, a list of key words, etc.)

Very important for search engines to find your site (SEO).

<style> Up to the developer

Style sheet (CSS) instructions. These instructions specify how the document should be formatted for display.

<script> Up to the developer

A code of client-side script in the document. Example languages are **JavaScript**.

HTML Document head <head>

<head>

```
<meta charset="utf-8" >  
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title> CSE211 Web programming </title>
```

```
<link rel="stylesheet" href="mystyles.css">
```

```
<style>  
..... css goes here  
</style>
```

```
<script src="script.js" defer>  
.....  
</script>
```

</head>

Arrangement of
head elements
isn't considered

HTML Document body

The **body element** contains the **actual content** of the page (what we want to show on the page). Consists of elements:

HTML Document body

Structural (Block-Level) Elements

- Define the main structure and layout.
- Organize content logically.
- Form the building blocks of a web page.
- Start on a new line.
- Can contain other elements
- E.g.: <header>, <main>, <article>, <aside>, <figure>, <nav>, <section>, <p>, ...

Text-Level (Inline) Elements

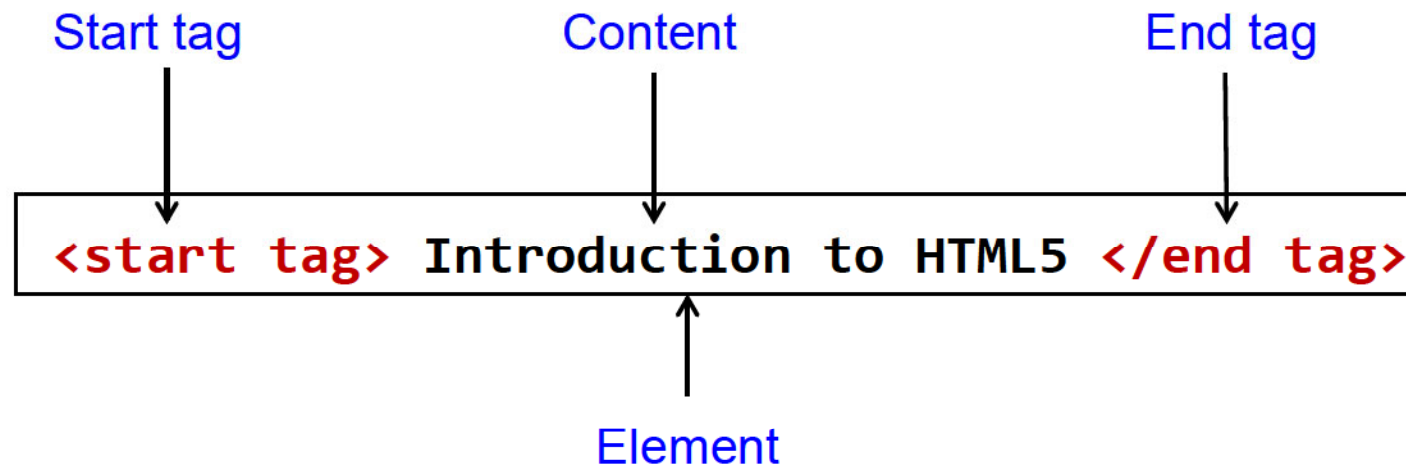
- Used within text to apply meaning, emphasis, or formatting.
- Don't break the paragraph flow.
- E.g. , , <mark>, , , <i>, <time>, <abbr>, <code> ...

Character-Level Elements & References

- Used for representing special symbols, non-ASCII characters, or reserved HTML characters.
- Don't breaking the paragraph flow
- Escape reserved characters (e.g., <, >, &)
- Display symbols not available on the keyboard (e.g., ©, °, ¥)
- E.G.: ©, €, <, &

HTML5 encourages semantic block elements for accessibility and SEO.

HTML5 Elements



- ❑ HTML5 element names by convention are written in **lowercase letters**.
- ❑ All elements, except for empty elements, consist of three pieces: a **start tag**, **content**, and an **end tag**.
- ❑ **Self-closing (void) elements** don't have content or an **end tag**.

E.g. `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`

Example: HTML elements

<p>

HTML is

awesome!!!

</p>

- An element usually has **start** and **end tags** (<p> and </p>)
 - **content**: stuff in between start and end tags
- Elements can be **nested**. Elements can **contain other elements** (p contains em and img)
- An element can be **self-closing** (img)
- An element can have **attributes** (src="puppy.jpg") belongs to the **start tag**.
 - *Attributes are name-value pairs that modify an element's behavior.*

1. Block-level elements

HTML5 Semantic Block Elements:

<header>	- Page/section header
<nav>	- Navigation links
<main>	- Primary content (ONE per page)
<article>	- Self-contained content
<section>	- Thematic grouping
<aside>	- Sidebar/tangential content
<footer>	- Page/section footer

Generic Containers (when semantic doesn't fit):

<div>	- generic block container
-------	---------------------------

Text containers

<p>	paragraph
<blockquote>	large quotation
<address>	address information

Headings

<h1>	level 1 header
<h2>	level 2 header
<h3>	level 3 header
<h4>	level 4 header
<h5>	level 5 header
<h6>	level 6 header

Lists

	unordered list
	ordered list
<dl>	definition list
<dt>	definition term
<dd>	data definition
<nav>	navigation bar

Data & Forms:

<form>	user input form
<table>	tables

2. Text-level elements

Semantic Text Markup (HTML5):

<code></code>	Emphasized text (stress emphasis)
<code></code>	Strong importance
<code><mark></code>	Highlighted/relevant text
<code><code></code>	Code snippet
<code><kbd></code>	Keyboard input
<code><samp></code>	Sample output
<code><var></code>	Variable
<code><cite></code>	Citation
<code><dfn></code>	Term definition
<code><abbr></code>	Abbreviation
<code><time></code>	Date/time

Forms (HTML5):

INPUT types: text, email, tel, url, number, date, color, range, search, password, file

<code><textarea></code>	- Multi-line text
<code><button></code>	- Clickable button
<code><fieldset></code>	- Group related inputs
<code><datalist></code>	- Input suggestions

Visual Styling (use sparingly, prefer CSS):

<code></code>	- Bold (no semantic meaning)
<code><i></code>	- Italic (no semantic meaning)
<code><u></code>	- Underline (confusing for links!)
<code><s></code>	- Strikethrough
<code><small></code>	- Fine print
<code><sup></code>	- Superscript
<code><sub></code>	- Subscript

<code><select></code>	- Dropdown list
<code><label></code>	- Input label (accessibility!)
<code><legend></code>	- Fieldset caption
<code><output></code>	- Calculation result

Headings and Paragraphs

◆ Heading Tags

```
<h1>Heading 1</h1>  
<h2>Sub heading 2</h2>  
<h3>Sub heading 3</h3>
```

Use headings (<h1> – <h6>) in hierarchical order for accessibility and SEO.

◆ Paragraph Tags

```
<p>This is my first paragraph</p>  
<p>This is my second paragraph</p>
```

Each <p> start on new line

◆ Sections: div and span

```
<div style="text-align:center; background: skyblue;">  
  This is a div  
</div>
```

<div> groups block content, while styles inline text.

headings.html

<head>

```
<meta charset="UTF-8">
```

<title>Headings and paragraphs</title>

</head>

<body>

Heading 1

Sub heading 2

Sub heading 3

This is my first paragraph

This is my second paragraph

```
<div text-align:center style="background:skyblue">
```

This is a div

</div>

</body>

</html>

Headings and paragraphs — Mozilla Firefox

File Edit View History Bookmarks Tools Help

Headings and paragraphs

00/headings.html

Heading 1

Sub heading 2

Sub heading 3

This is my first paragraph

This is my second paragraph

This is a div

Heading 1

Sub heading 2

Sub heading 3

This is my first paragraph

This is my second paragraph

This is a div

Browsers apply default CSS (user agent styles) to elements like headings and paragraphs; we'll customize these soon with CSS

Text Styling without CSS

- ◆ Text can be formatted as headings or regular paragraph text
 - ◆ Visual differences between headings are controlled by browser default styles (font-size, weight, margins)
- ◆ `<p> </p>` by default doubles the spaces after each paragraph
- ◆ `<br>` and `<hr>` are empty elements; no closing tags.
- ◆ Different styles of heading are available:

<code><h1></h1></code>	Heading 1
<code><h2></h2></code>	Heading 2
<code><h3></h3></code>	Heading 3
<code><h4></h4></code>	Heading 4
<code><h5></h5></code>	Heading 5
<code><h6></h6></code>	Heading 6

Text Formatting (level) Elements

- ❑ Text formatting elements are referred to as **Text-level elements** because, unlike the block-level elements, which describe blocks of content, **These elements apply formatting or meaning to inline text fragments rather than entire blocks.**
- ❑ Character-level elements **describe the formatting of words or phrases** as opposed to sections or paragraphs.
- ❑ There are two basic groups of text formatting elements: **presentation styles**, and **logical styles**.
 - **Presentation styles** describe how the text should be **displayed**, in **bold** type or *italics*, for example.
 - **Logical styles** describe the **meaning** of the style more than the actual format.
 - Modern HTML5 favors logical styles (semantic meaning) over pure presentation.

Text Formatting

- ◆ Text formatting tags modify the text between the opening tag and the closing tag
 - ◆ Ex. `<strong> Hello </strong>` makes “Hello” bold

<code><u></u></code>	<u>underlined</u>
<code><sup></sup></code>	Sample ^{superscript}
<code><sub></sub></code>	Sample _{subscript}
<code></code>	strong
<code></code>	<i>emphasized</i>
<code><pre></pre></code>	Preformatted text
<code><blockquote></blockquote></code>	Quoted text block
<code></code>	Deleted text – strike through
<code><ins></ins></code>	inserted text, underlined by default
<code><mark> ... </mark></code>	Highlighted text

Logical Text Formatting Elements

- ❑ Logical text formatting describe the **meaning of the style** more than the actual format.

Element Name	Formatting Style
<cite>...</cite>	Defines a citation
<code>...</code>	Presents computer code examples
...	In most browsers, this is italics
...	In most browsers, this is bold
...	Provides a logical inline grouping with no predefined look.
<dfn>	element indicates a term to be defined
<abbr>	element represents an abbreviation or acronym
<var>	element represents the name of a variable in a mathematical expression or a programming context

Character Entities In HTML

- Certain characters in HTML have special meaning to parser (e.g., ©, °, ¥)
- To add these characters as literal values, use the **character entity code** (**&code;**).
 - It begins with an ampersand (**&**) character, then the **code for the entity**, then a semicolon (**;**), (e.g. © € < &)
 - **Character code** is referenced using their decimal value in the **ASCII** or UTF encoding tables.

Symbol	Description	HTML Code
>	Greater than	> or &62;
<	Less than	< or &60;
®	Trademark	™ or &174;
©	Copyright	© or &169;
¢	Cent sign	¢ or &162;
	Non-breaking space	 or &160

Logical Text Formatting – Example

Logical (Semantic) Text Formatting — Mozilla Firefox

File Edit View History Bookmarks Tools Help

Logical (Semantic) Text Formatting × +

← → ↻ 🏠 🛡️ 📄 http://127.0.0.1:5500/ 🔍 Search ⬇️ ||| 📄 >> ☰

Logical (Semantic) Text Formatting

The following is a citation using

“Four score and seven years ago our fathers brought forth on this continent, a new nation, conceived in Liberty, and dedicated to the proposition that all men are created equal.”
— *Gettysburg Address*

The following is a block of code using `<pre>` and `<code>`:

```
// Count up to 10
let count = 0;
while (count < 10) {
  count++;
}
console.log(count); // 10
```

The last two words of this sentence are marked with **strong importance**.

The last two words of this sentence are given *emphasis semantics*.

Extra examples: an API, a variable x , and a highlighted phrase.

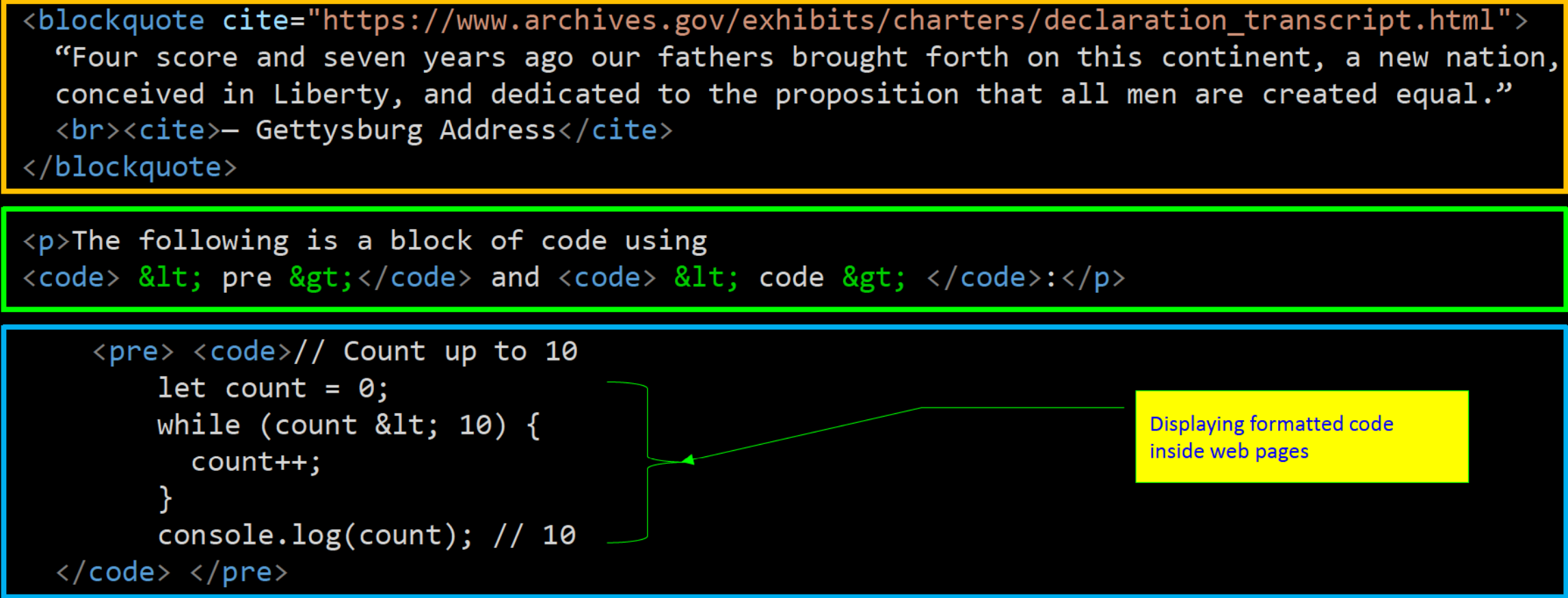
Logical Text Formatting – Example

```
<body>
  <main>
    <h1>Logical (Semantic) Text Formatting</h1>
    <p>The following is a citation using </p>
    <blockquote cite="https://www.archives.gov/exhibits/charters/declaration_transcript.html">
      "Four score and seven years ago our fathers brought forth on this continent, a new nation,
      conceived in Liberty, and dedicated to the proposition that all men are created equal."
      <br><cite>– Gettysburg Address</cite>
    </blockquote>

    <p>The following is a block of code using
    <code> &lt; pre &gt; and <code> &lt; code &gt; </code>:</p>

    <pre> <code>// Count up to 10
      let count = 0;
      while (count &lt; 10) {
        count++;
      }
      console.log(count); // 10
    </code> </pre>

  </main>
</body>
```



Displaying formatted code inside web pages

Logical Text Formatting – Example

```
<body>
  <main>
    <p> The last two words of this sentence are marked with
      <strong>strong importance</strong>. </p>
    <p> The last two words of this sentence are given
      <em>emphasis semantics</em>.
    </p>
    <p> Extra examples: an
      <abbr title="Application Programming Interface">API</abbr>,
      a variable <var> x </var>, and a <mark>highlighted phrase</mark>.
    </p>
  </main>
</body>
```

Attributes

- HTML5 **attributes** are pieces of information that help to describe elements.
- Some elements have **required attributes**; others are **optional** and depend on the content that is being marked up.
- Attributes are referred to as **name-value pairs**

name = "value"

examples:

**** Click here **** *to add hyperlink*

**** *to add image*

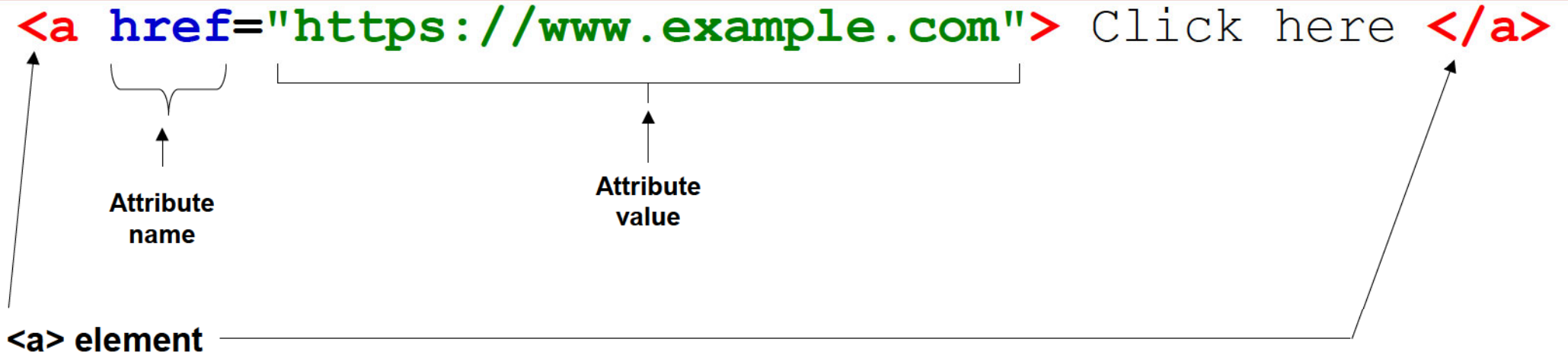
- **Special Case: Boolean Attributes (attributes without values)**

No value needed (or use attribute name as value)

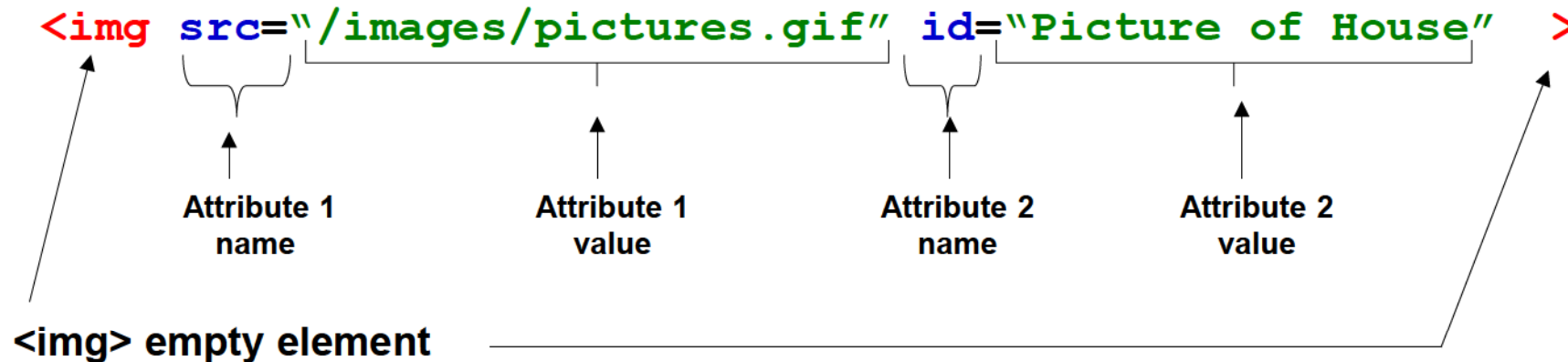
E,g. **<button disabled>** Click Me **</button>**

Attributes & Attribute Rules

Example:



Example:



Example:

`<img`
 `src="images/house.jpg"`
 `alt="Front view of a modern house"`
 `id="houseImage"`
 `width="400"`
 `height="250" >`

Some elements use more
than one attribute

Hands-On 1 — Basic HTML Structure

Goal: Practice writing a complete HTML page with heading and paragraph tags.

Task:

Create a webpage that includes:

- Add a <title> tag and a comment describing the page purpose.
- A main heading (<h1>)
- Two paragraphs (<p>)
- A horizontal line (<hr>)
- A comment line

❑ Save the file as exercise1.html

Questions?

